

Big Data Analytics (PETR 6397 21860)

Midterm 1

Greyson Newton

```
In [14]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import math

def load_data(file_path: str) -> pd.DataFrame:
    df = pd.read_csv(file_path)
    print(f"Dataframe shape: {df.shape}")
    print(f"Dataframe column types: {df.dtypes}")
    return df

df = load_data("data_midterm.csv")
```

```
Dataframe shape: (200, 7)
Dataframe column types: Depth (ft)                                float64
Porosity (%)                                                    float64
Matrix Perm (nd)                                                int64
Acoustic impedance (kg/m2s*10^6)                                float64
Brittleness Ratio                                              float64
TOC (%)                                                         object
Vitrinite Reflectance (%)                                       object
dtype: object
```

1. Remove all samples with missing measurements and report the number of removed samples.

removed 9 samples with missing measurements (Null or 'na' values)

```
In [15]: def remove_nulls(df: pd.DataFrame) -> pd.DataFrame:

    old_len = len(df)
    df = df.replace('na', np.nan).dropna()
    num_removed_nulls = old_len - len(df)
    print(f"Number of rows removed: {num_removed_nulls}")
    return df

df = remove_nulls(df)
```

```
df[['TOC (%)']] = df[['TOC (%)']].astype(np.float32)
df[['Vitrinite Reflectance (%)']] = df[['Vitrinite Reflectance (%)']].astype(
df.dtypes
```

Number of rows removed: 9

```
Out[15]: Depth (ft)                float64
Porosity (%)                    float64
Matrix Perm (nd)                int64
Acoustic impedance (kg/m2s*10^6) float64
Brittleness Ratio              float64
TOC (%)                        float32
Vitrinite Reflectance (%)      float32
dtype: object
```

2. Check for duplicate entries and remove them if they are errors. Report the number of samples removed.

0 samples removed

```
In [16]: def remove_duplicates(df:pd.DataFrame)->pd.DataFrame:
old_len = len(df)
df = df.drop_duplicates()
num_removed_duplicates = old_len - len(df)
print(f"Number of rows removed: {num_removed_duplicates}")
return df

df = remove_duplicates(df)
```

Number of rows removed: 0

3. Identify and remove samples with outlier porosity values. Show how the outliers were determined and report the number of samples removed.

Outlier Removal via Z-Score Filtering

Given a dataset with a variable X (Porosity (%)), outlier removal is performed as follows:

- Calculate the mean μ and standard deviation σ of X .
- For each observation x_i , compute its z-score: $z_i = \frac{x_i - \mu}{\sigma}$.
- Determine the absolute value of each z-score, $|z_i|$.
- Define an outlier threshold (e.g., 2). An observation is considered an outlier if $|z_i| \geq 2$.
- Remove any observations where $|z_i| \geq 2$; retain those with $|z_i| < 2$.

Threshold = 1

- Number of rows removed: 65
- Percentage of dataset removed: 34.03%

Threshold = 2

- Number of rows removed: 4
- Percentage of dataset removed: 2.09%

Threshold = 3

- Number of rows removed: 0
- Percentage of dataset removed: 0.00%

I used a threshold of 2 for this dataset.

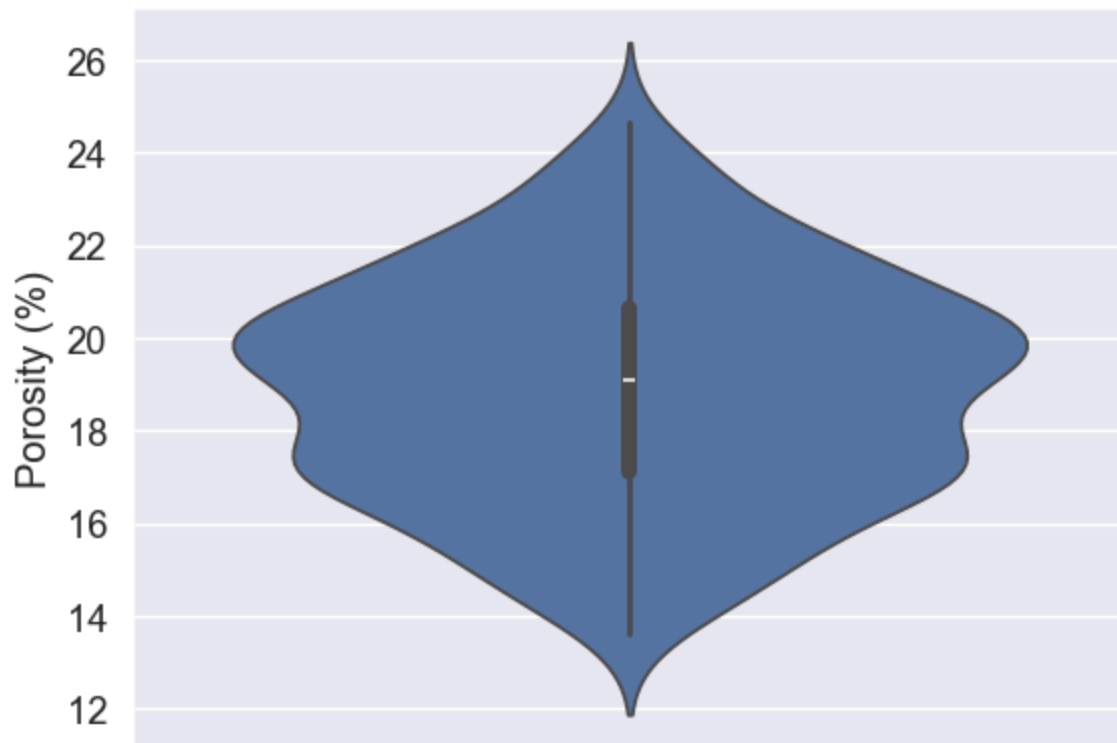
In [17]: *# apply pca*

```
In [18]: # z score outlier removal
def remove_outliers(df: pd.DataFrame, column_name: str, threshold: int = 3)
    old_len = len(df)
    series = df[column_name]
    z_scores = (series - series.mean()) / series.std()
    abs_z_scores = np.abs(z_scores)
    filtered_entries = (abs_z_scores < threshold)
    df_filtered = df[filtered_entries]
    new_len = len(df_filtered)
    percent_removed = (old_len - new_len) / old_len * 100 # percent removed
    print(f"Number of rows removed: {old_len - new_len}")
    print(f"Percentage of dataset removed: {percent_removed:.2f}%")
    return df_filtered

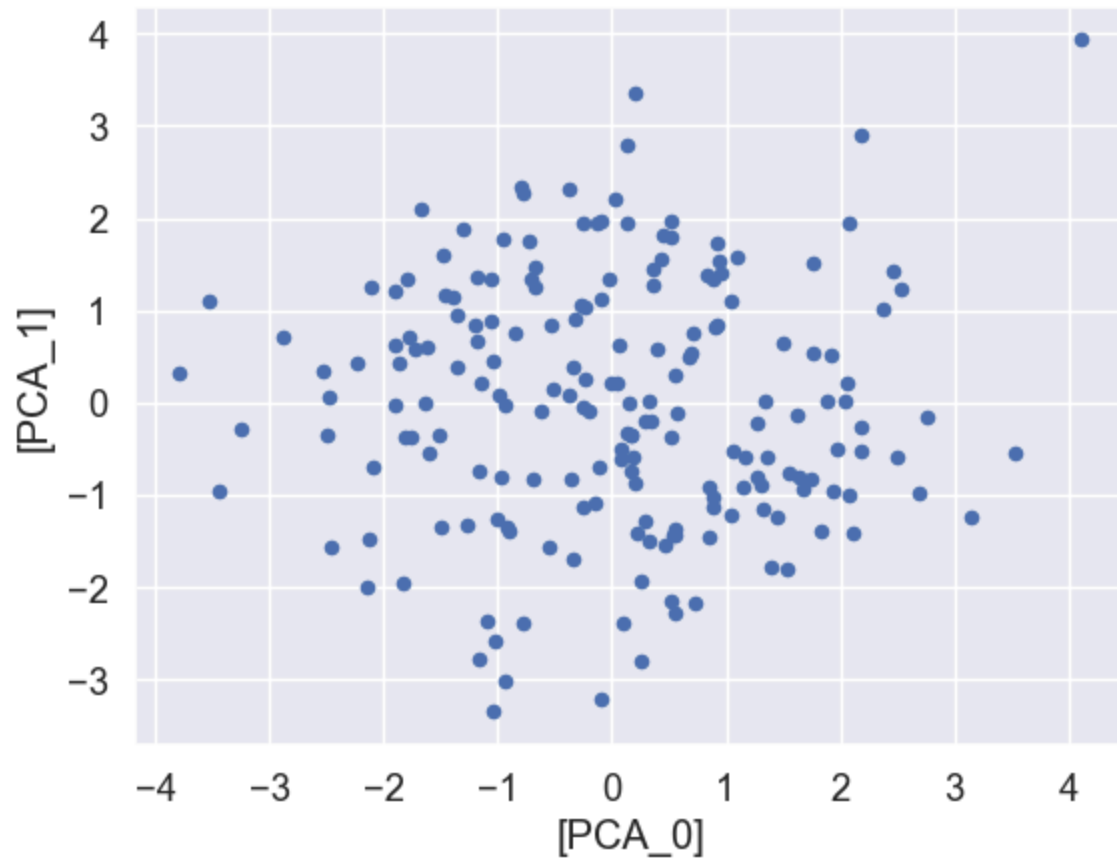
#violin plot
def plot_violin(df:pd.DataFrame, column_name:str):
    sns.violinplot(y=column_name, data=df)
    plt.show()

from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
# plot pca
def plot_pca(df: pd.DataFrame, n_components: int) -> pd.DataFrame:
    scaler = StandardScaler()
    pca = PCA(n_components=n_components)
    pipeline = make_pipeline(scaler, pca)
    pipeline.fit(df)
    pca_features = pipeline.transform(df)
    pca_features_df = pd.DataFrame(pca_features)
    pca_features_df.columns = [f"PCA_{i}" for i in range(n_components)]
    print(f"PCA features shape: {pca_features_df.shape}")
    pca_features_df.plot(kind='scatter', x=['PCA_0'], y=['PCA_1'])
    plt.show()
```

```
plot_violin(df, 'Porosity (%)')  
plot_pca(df, 2)  
df = remove_outliers(df, 'Porosity (%)', 2)  
plot_violin(df, 'Porosity (%)')  
plot_pca(df, 2)
```

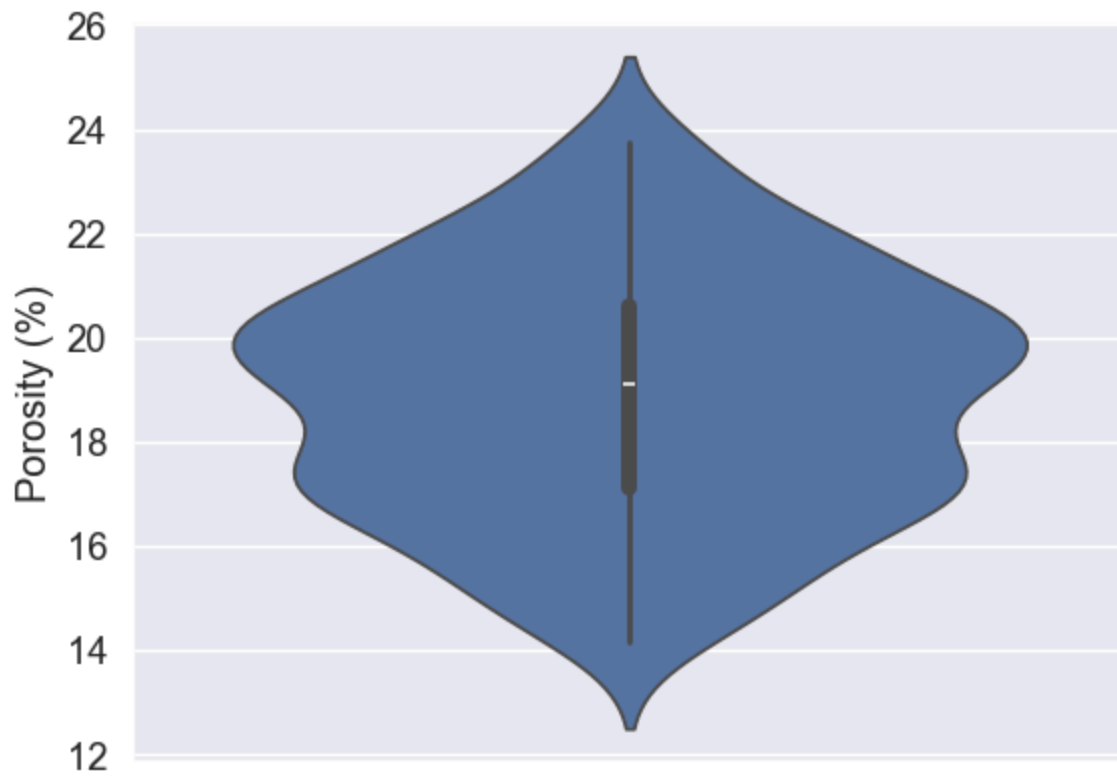


PCA features shape: (191, 2)

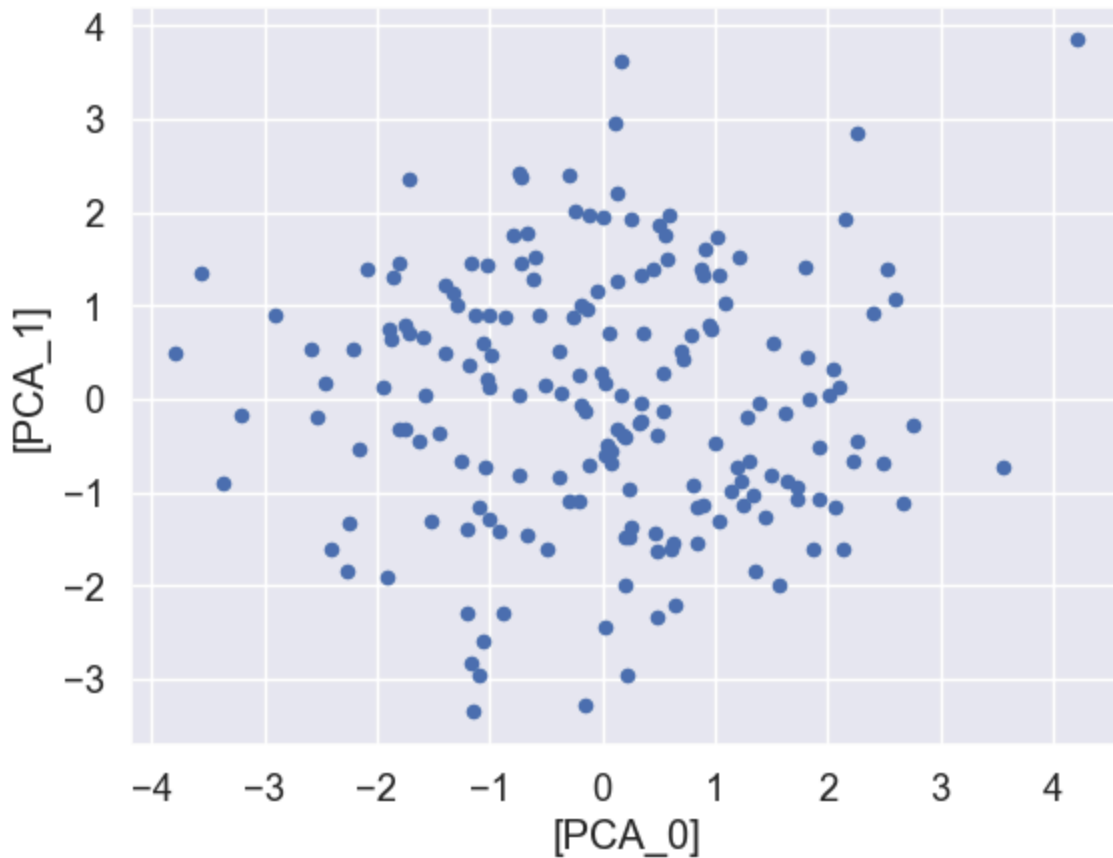


Number of rows removed: 4

Percentage of dataset removed: 2.09%



PCA features shape: (187, 2)



4. Present the cross plots and histograms of all the cleaned properties.

```
In [19]: # Crossplot with reg. lines.
def plot_cross(df: pd.DataFrame):
    # initialize r2 mat
    num_cols = len(df.columns)
    r2_matrix = np.empty((num_cols, num_cols))
    r2_matrix[:] = np.nan

    fig, axs = plt.subplots(num_cols, num_cols, figsize=(15,15))
    for i in range(num_cols):
        for j in range(num_cols):
            ax = axs[i][j]
            if i == j:
                # remove diagonals
                ax.set_visible(False)
            else:
                # scatter non-diagonals
                ax.scatter(df[df.columns[i]], df[df.columns[j]], s=10)

                # fit basic linear reg
                x = df[df.columns[i]]
                y = df[df.columns[j]]
                slope, intercept = np.polyfit(x, y, 1)
```

```

# plot linear reg line
x_vals = np.array(ax.get_xlim())
y_vals = intercept + slope * x_vals
ax.plot(x_vals, y_vals, '--', color='red')

# r2 calculation
y_pred = intercept + slope * x
ss_res = np.sum((y - y_pred)**2)
ss_tot = np.sum((y - y.mean())**2)
r2 = 1 - (ss_res/ss_tot) if ss_tot != 0 else np.nan
r2_matrix[i, j] = r2

# labels
if j == 0:
    ax.set_ylabel(df.columns[i], fontsize=8)
if i == num_cols - 1:
    ax.set_xlabel(df.columns[j], fontsize=8)

plt.tight_layout()
plt.show()
return r2_matrix

# r2 plot
def plot_r2_heatmap(r2_matrix: np.ndarray, columns: pd.Index):
    plt.figure(figsize=(10, 10))
    sns.set(font_scale=1.2)
    sns.heatmap(r2_matrix, annot=True, xticklabels=columns, yticklabels=columns)
    plt.title("R2 Matrix Heatmap")
    plt.show()

# value count plot
def plot_value_counts(df: pd.DataFrame, max_cols:int = 3):
    num_features = len(df.columns)
    ncols = min(max_cols, num_features)
    nrows = math.ceil(num_features / ncols)

    fig, axs = plt.subplots(nrows, ncols, figsize=(5*ncols, 4*nrows))
    axs = axs.flatten() if num_features > 1 else [axs]

    for i, col in enumerate(df.columns):
        if pd.api.types.is_numeric_dtype(df[col]):
            data = df[col].dropna()
            # Plot density histogram
            axs[i].hist(data, bins='auto', density=True, color='skyblue', ec='black')
            # Plot vertical red dashed line for mean
            mean_val = data.mean()
            axs[i].axvline(mean_val, color='red', linestyle='--', linewidth=2)
            # Overlay PDF using kernel density estimate
            sns.kdeplot(data, ax=axs[i], color='black', lw=2)
            axs[i].set_title(f"{col} (Histogram & PDF)")
            axs[i].set_xlabel("Value")
            axs[i].set_ylabel("Density")
            axs[i].legend()
        else:
            # For non-numeric columns, plot value counts
            counts = df[col].value_counts().sort_index()

```

```

    axs[i].bar(counts.index.astype(str), counts.values, color='skybl
    axs[i].set_title(f"{col} (Value Counts)")
    axs[i].set_xlabel("Value")
    axs[i].set_ylabel("Count")
    axs[i].tick_params(axis='x', rotation=45)

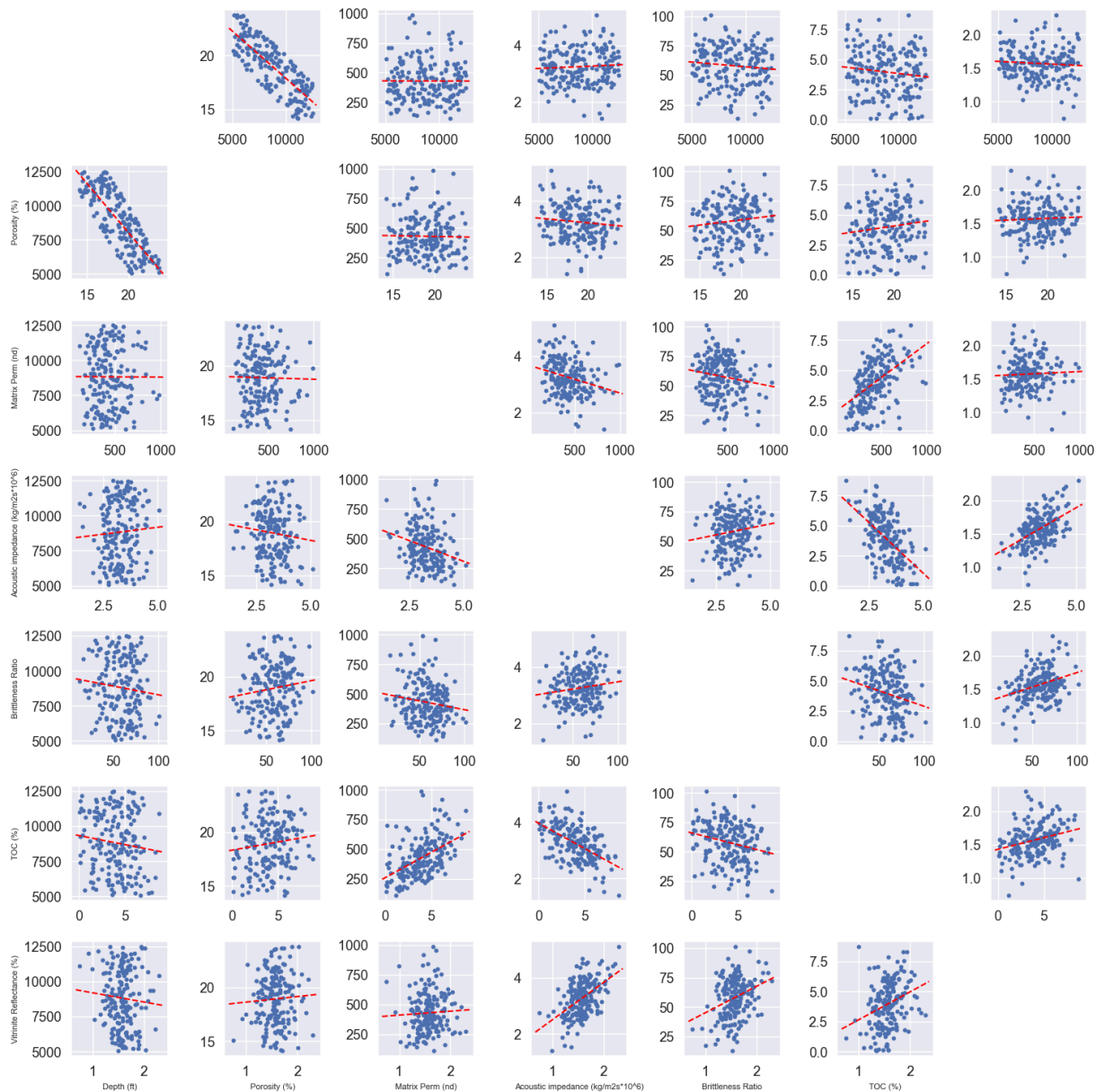
# Hide any unused subplots
for j in range(i+1, len(axs)):
    axs[j].axis('off')

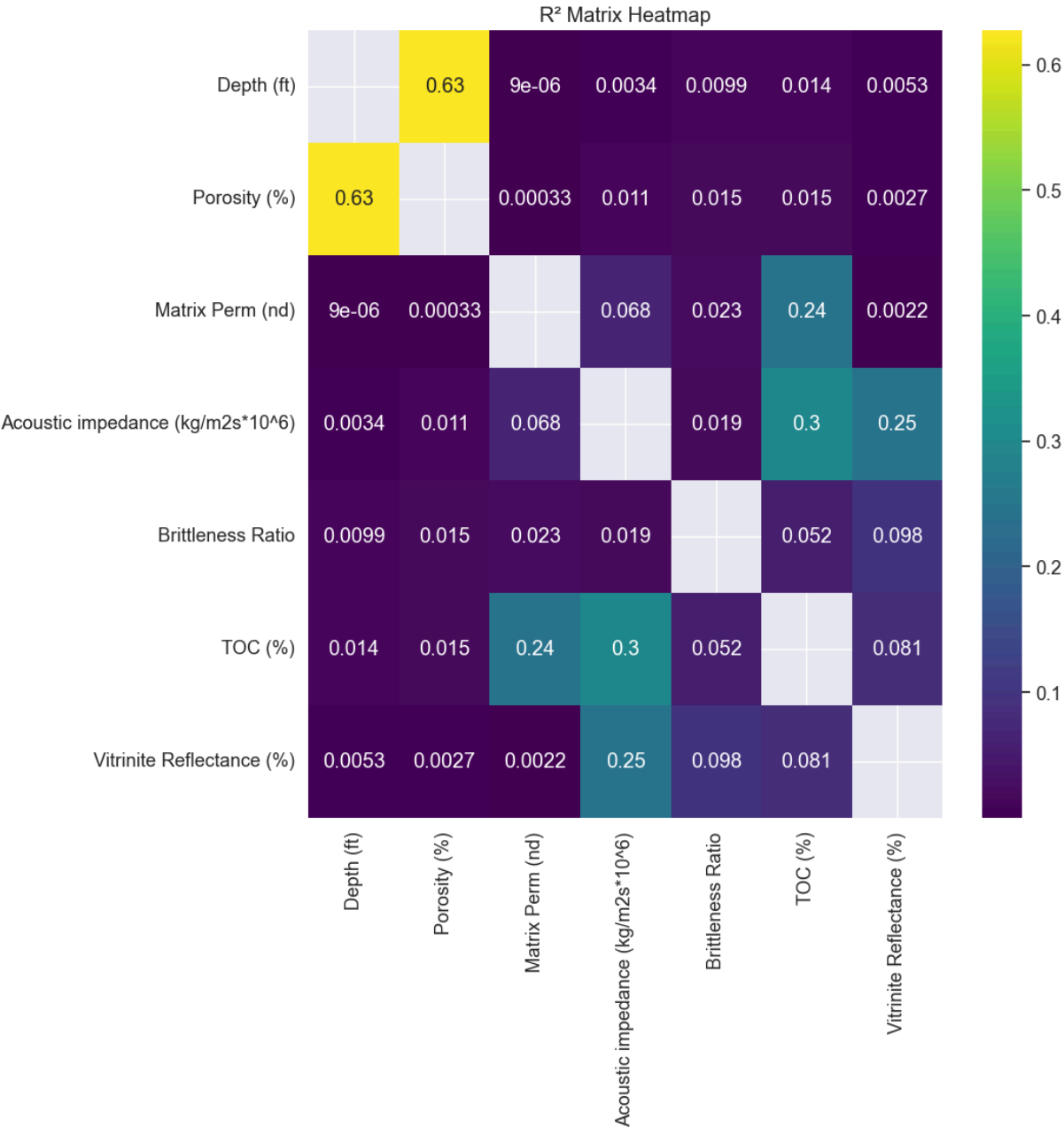
plt.tight_layout()
plt.show()

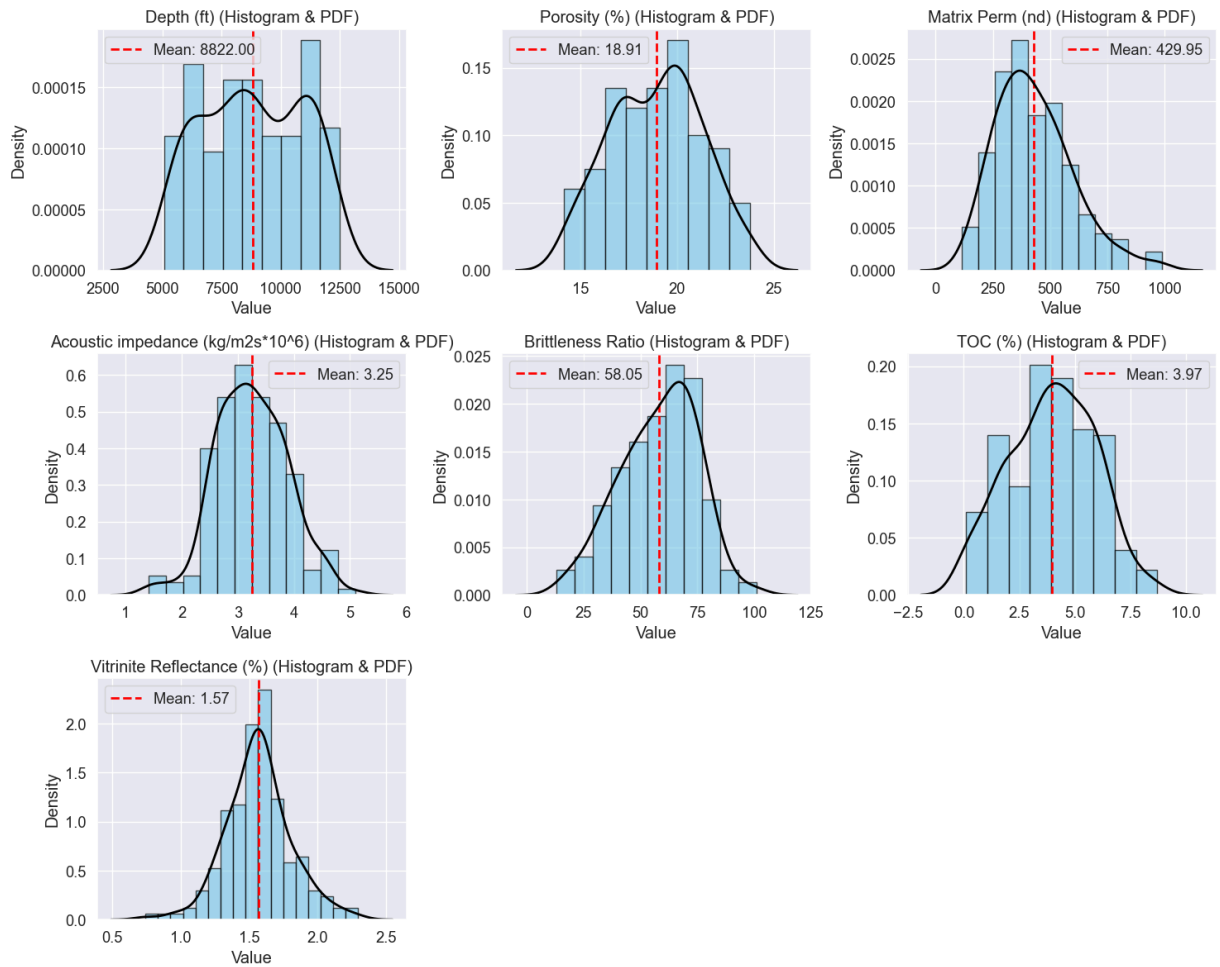
# Assuming your DataFrame 'df' is already loaded:
r2_matrix = plot_cross(df)
plot_r2_heatmap(r2_matrix, df.columns)
plot_value_counts(df)

# Optionally, save the cleaned DataFrame to CSV
df.to_csv("cleaned_data.csv", index=False)

```





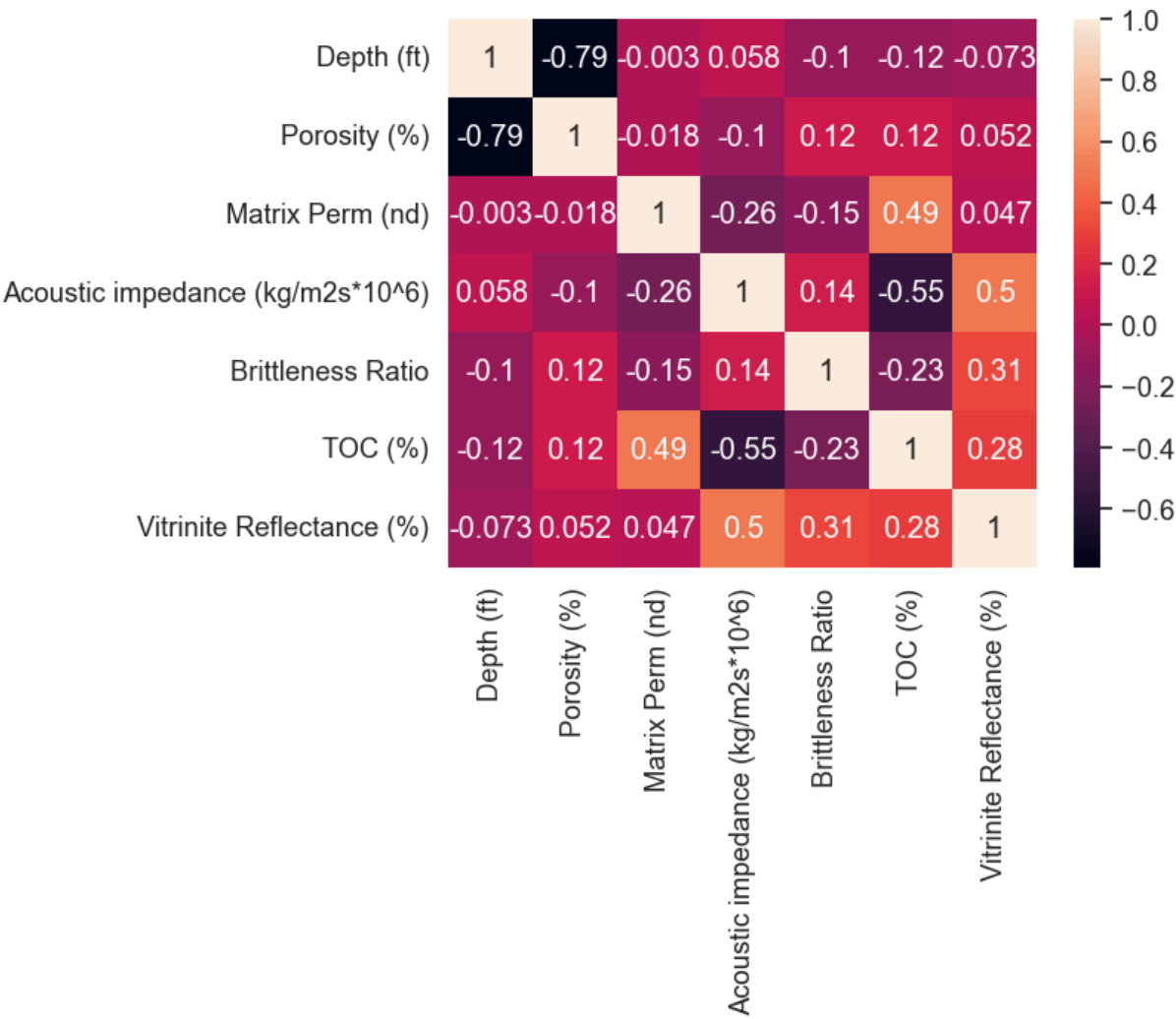


5. Determine all the correlation coefficients.

correlation table found below:

```
In [20]: # determine and plot all the correlation coefficients
def get_correlation_matrix(df:pd.DataFrame)->pd.DataFrame:
    corr = df.corr()
    sns.heatmap(corr, annot=True)
    plt.show()
    return corr

corr = get_correlation_matrix(df)
print(corr)
```



\	Depth (ft)	Porosity (%)	Matrix Perm (nd)
Depth (ft)	1.000000	-0.792233	-0.003006
Porosity (%)	-0.792233	1.000000	-0.018049
Matrix Perm (nd)	-0.003006	-0.018049	1.000000
Acoustic impedance (kg/m2s*10^6)	0.058110	-0.104281	-0.261163
Brittleness Ratio	-0.099737	0.122497	-0.151903
TOC (%)	-0.117419	0.124296	0.489513
Vitrinite Reflectance (%)	-0.073122	0.051905	0.046908

	Acoustic impedance (kg/m2s*10^6)	\
Depth (ft)	0.058110	
Porosity (%)	-0.104281	
Matrix Perm (nd)	-0.261163	
Acoustic impedance (kg/m2s*10^6)	1.000000	
Brittleness Ratio	0.138370	
TOC (%)	-0.548278	
Vitrinite Reflectance (%)	0.496412	

	Brittleness Ratio	TOC (%)	\
Depth (ft)	-0.099737	-0.117419	
Porosity (%)	0.122497	0.124296	
Matrix Perm (nd)	-0.151903	0.489513	
Acoustic impedance (kg/m2s*10^6)	0.138370	-0.548278	
Brittleness Ratio	1.000000	-0.227365	
TOC (%)	-0.227365	1.000000	
Vitrinite Reflectance (%)	0.312634	0.283907	

	Vitrinite Reflectance (%)
Depth (ft)	-0.073122
Porosity (%)	0.051905
Matrix Perm (nd)	0.046908
Acoustic impedance (kg/m2s*10^6)	0.496412
Brittleness Ratio	0.312634
TOC (%)	0.283907
Vitrinite Reflectance (%)	1.000000

6. Discuss whether the correlation coefficients capture the dependency in this dataset.

The correlation coefficients capture several dependencies in the dataset.

However, the target itself (**Porosity**) is particularly dependent on 1 feature **Depth (ft)** based on the correlation coefficients.

7. Use a regression method of your choice to determine porosity as a function of depth. Report R2 and MSE.

Based on the results of training multiple regression models, the model that I would select is **Support Vector Machine**:

math
 $r^2 = 0.62$, MSE = 0.63

```
In [21]: import yaml
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.model_selection import train_test_split, GridSearchCV
import numpy as np
import pandas as pd

# holds a registry of different regression models
class RegressionFactory:
    def __init__(self):
        '''Registry for modls'''
        self.models = {
            'linear': LinearRegression(),
            'ridge': Ridge(),
            'lasso': Lasso(),
            'elastic': ElasticNet(),
            'Random Forest': RandomForestRegressor(n_estimators=100, random_
            'Gradient Boosting': GradientBoostingRegressor(n_estimators=100,
            'Support Vector Regressor': SVR()
        }
        self.metrics = {}
    def register(self, name, model):
        self.models[name] = model
    def get(self, name):
        return self.models.get(name)
    def register_metrics(self, name, metrics):
        self.metrics[name] = metrics

# PipelineFactory that builds a pipeline with scaling and modeling.
class PipelineFactory:
    def __init__(self):
        '''Registry for pipelines'''
        self.pipelines = {}
    def create_pipeline(self, model):
        '''Creates a pipeline with scaling and the given model.'''
        return Pipeline([
            ('scaler', StandardScaler()),
            ('model', model)
        ])
    def register(self, name, pipeline):
        self.pipelines[name] = pipeline
    def get(self, name):
        return self.pipelines.get(name)

def split_data(df: pd.DataFrame, target: str, features: list, **kwargs) -> c
```

```

train, test = train_test_split(df, **kwargs)
if features is None:
    features = df.drop(target, axis=1).columns.tolist()
train_X = train[features]
train_y = train[target]
test_X = test[features]
test_y = test[target]
return {'train_X': train_X, 'train_y': train_y, 'test_X': test_X, 'test_y': test_y}

def train_pipelines(
    df: pd.DataFrame,
    target: str,
    features: list,
    model_names: list,
    model_factory: RegressionFactory,
    pipeline_factory: PipelineFactory,
    param_grids: dict = {}
) -> None:

    # get data
    dataset = split_data(df, target, features, test_size=0.2, random_state=42)

    # train pipelines
    for model_name in model_names:
        # instantiate pipeline
        model = model_factory.get(model_name)
        pipeline = pipeline_factory.create_pipeline(model)
        grid_params = param_grids.get(model_name, {})
        # tune if tunable, else regular fit
        if grid_params:
            gs = GridSearchCV(pipeline, grid_params, cv=5)
            gs.fit(dataset['train_X'], dataset['train_y'])
            pipeline = gs.best_estimator_
            print(f"{model_name} pipeline tuned successfully, best parameters: {gs.best_params_}")
        else:
            pipeline.fit(dataset['train_X'], dataset['train_y'])
            print(f"{model_name} pipeline trained successfully (no tuning parameters)")
        # evaluate and register
        y_pred = pipeline.predict(dataset['test_X'])
        r2 = pipeline.score(dataset['test_X'], dataset['test_y'])
        mse = np.mean((y_pred - dataset['test_y'])**2)
        # Register pipeline and metrics
        pipeline_factory.register(model_name, pipeline)
        model_factory.register_metrics(model_name, {'r2': r2, 'mse': mse})

```

no tuning

```

In [22]: # config
param_grids = {}
model_factory = RegressionFactory()
pipeline_factory = PipelineFactory()
target = 'Porosity (%)'
features = ['Depth (ft)']
models = ['linear', 'ridge', 'lasso', 'elastic', 'Random Forest',

```

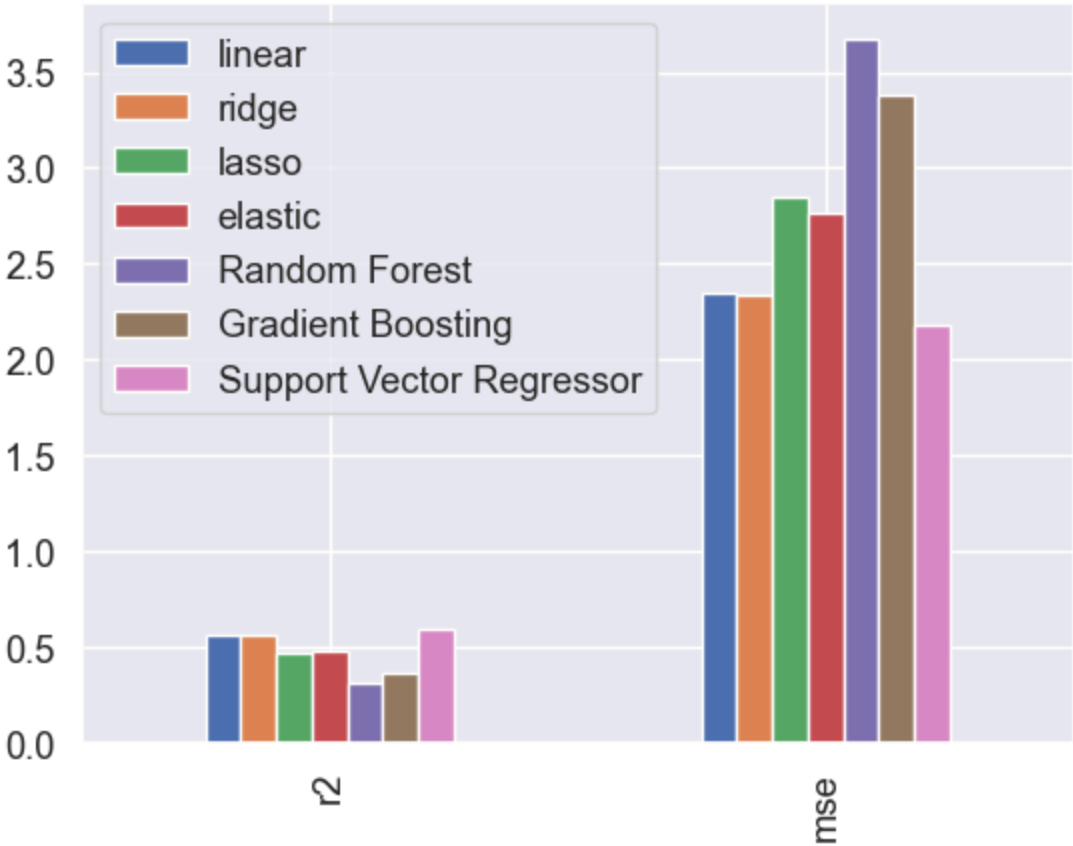
```
                'Gradient Boosting', 'Support Vector Regressor']
# trian
train_pipelines(df, target, features, models, model_factory,
                pipeline_factory, param_grids)

# visualize
metrics_df = pd.DataFrame(model_factory.metrics)
metrics_df.plot(kind='bar')
metrics_df
```

linear pipeline trained successfully (no tuning parameters)
ridge pipeline trained successfully (no tuning parameters)
lasso pipeline trained successfully (no tuning parameters)
elastic pipeline trained successfully (no tuning parameters)
Random Forest pipeline trained successfully (no tuning parameters)
Gradient Boosting pipeline trained successfully (no tuning parameters)
Support Vector Regressor pipeline trained successfully (no tuning parameter
s)

Out [22]:

	linear	ridge	lasso	elastic	Random Forest	Gradient Boosting	Support Vector Regressor
r2	0.562606	0.564567	0.469153	0.483467	0.314714	0.369060	0.592612
mse	2.343889	2.333379	2.844679	2.767973	3.672276	3.381054	2.183091



tuning

tuning didnt really help the models that were already performing well with the default parameters.

```
In [23]: # config
with open("param_grids.yml", "r") as f:
    param_grids = yaml.safe_load(f)

model_factory = RegressionFactory()
pipeline_factory = PipelineFactory()
target = 'Porosity (%)'
features = ['Depth (ft)']
models = ['linear', 'ridge', 'lasso', 'elastic', 'Random Forest',
          'Gradient Boosting', 'Support Vector Regressor']

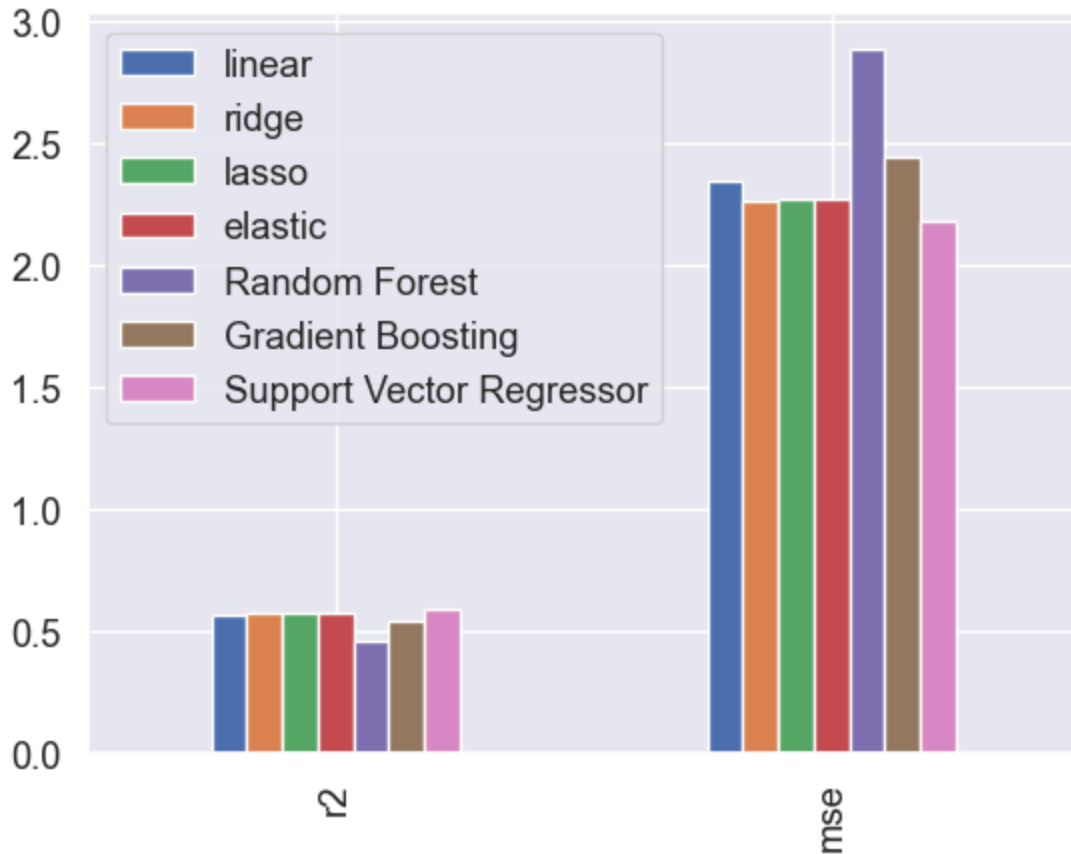
# train
train_pipelines(df, target, features, models, model_factory,
                pipeline_factory, param_grids)

# visualize
metrics_df = pd.DataFrame(model_factory.metrics)
metrics_df.plot(kind='bar')
metrics_df
```

linear pipeline trained successfully (no tuning parameters)
 ridge pipeline tuned successfully, best parameters: {'model__alpha': 10.0}
 lasso pipeline tuned successfully, best parameters: {'model__alpha': 0.1}
 elastic pipeline tuned successfully, best parameters: {'model__alpha': 0.1, 'model__l1_ratio': 0.9}
 Random Forest pipeline tuned successfully, best parameters: {'model__max_depth': 5, 'model__n_estimators': 200}
 Gradient Boosting pipeline tuned successfully, best parameters: {'model__learning_rate': 0.01, 'model__n_estimators': 200}
 Support Vector Regressor pipeline tuned successfully, best parameters: {'model__C': 1.0, 'model__gamma': 'scale'}

Out [23]:

	linear	ridge	lasso	elastic	Random Forest	Gradient Boosting	Support Vector Regressor
r2	0.562606	0.577962	0.576097	0.576920	0.461594	0.544136	0.592612
mse	2.343889	2.261595	2.271590	2.267181	2.885183	2.442865	2.183091



8. Determine if porosity shows a stronger dependency on any other property than indicated in problem 7. If so, develop an alternative regression model and report its R^2 and MSE values.

The results of training multiple tuned regression models for each possible predictor indicates that there is no stronger dependency on any other property for estimating porosity. This corroborates EDA results that showed **Depth (ft)** as the most correlated feature with **Porosity**.

Feature Importance results are found below.

```
In [24]: with open("param_grids.yml", "r") as f:
          param_grids = yaml.safe_load(f)

# Instantiate factories and train pipelines with tuning functionality
model_factory = RegressionFactory()
pipeline_factory = PipelineFactory()
target = 'Porosity (%)'
feature_set = [[feature] for feature in df.drop(target,axis=1).columns]
models = ['linear', 'ridge', 'lasso', 'elastic', 'Random Forest',
          'Gradient Boosting', 'Support Vector Regressor']
```

```
best_features = dict()
for features in feature_set:

    train_pipelines(df, target, features, models, model_factory,
                    pipeline_factory, param_grids)

    best_model = max(model_factory.metrics, key=lambda x: model_factory.metrics[x])

    best_features[features[0]] = {
        'r2': model_factory.metrics[best_model]['r2'],
        'mse': model_factory.metrics[best_model]['mse'],
        'model': best_model
    }
    print(f"Best model for feature set {features}: {best_model} with R^2: {model_factory.metrics[best_model]['r2']}")
    # Visualize the metrics with a bar plot

best_features
```

```
linear pipeline trained successfully (no tuning parameters)
ridge pipeline tuned successfully, best parameters: {'model__alpha': 10.0}
lasso pipeline tuned successfully, best parameters: {'model__alpha': 0.1}
elastic pipeline tuned successfully, best parameters: {'model__alpha': 0.1,
'model__l1_ratio': 0.9}
Random Forest pipeline tuned successfully, best parameters: {'model__max_dep
th': 5, 'model__n_estimators': 200}
Gradient Boosting pipeline tuned successfully, best parameters: {'model__lea
rning_rate': 0.01, 'model__n_estimators': 200}
Support Vector Regressor pipeline tuned successfully, best parameters: {'mod
el__C': 1.0, 'model__gamma': 'scale'}
Best model for feature set ['Depth (ft)']: Support Vector Regressor with R^
2: 0.5926121023161295
linear pipeline trained successfully (no tuning parameters)
ridge pipeline tuned successfully, best parameters: {'model__alpha': 10.0}
lasso pipeline tuned successfully, best parameters: {'model__alpha': 1.0}
elastic pipeline tuned successfully, best parameters: {'model__alpha': 1.0,
'model__l1_ratio': 0.5}
Random Forest pipeline tuned successfully, best parameters: {'model__max_dep
th': 5, 'model__n_estimators': 200}
Gradient Boosting pipeline tuned successfully, best parameters: {'model__lea
rning_rate': 0.01, 'model__n_estimators': 50}
Support Vector Regressor pipeline tuned successfully, best parameters: {'mod
el__C': 0.1, 'model__gamma': 'auto'}
Best model for feature set ['Matrix Perm (nd)']: linear with R^2: -0.0261830
29275428815
linear pipeline trained successfully (no tuning parameters)
ridge pipeline tuned successfully, best parameters: {'model__alpha': 0.1}
lasso pipeline tuned successfully, best parameters: {'model__alpha': 0.001}
elastic pipeline tuned successfully, best parameters: {'model__alpha': 0.00
1, 'model__l1_ratio': 0.1}
Random Forest pipeline tuned successfully, best parameters: {'model__max_dep
th': 5, 'model__n_estimators': 50}
Gradient Boosting pipeline tuned successfully, best parameters: {'model__lea
rning_rate': 0.01, 'model__n_estimators': 50}
Support Vector Regressor pipeline tuned successfully, best parameters: {'mod
el__C': 0.1, 'model__gamma': 'scale'}
Best model for feature set ['Acoustic impedance (kg/m2s*10^6)']: lasso with
R^2: -0.035675584897860046
linear pipeline trained successfully (no tuning parameters)
ridge pipeline tuned successfully, best parameters: {'model__alpha': 10.0}
lasso pipeline tuned successfully, best parameters: {'model__alpha': 1.0}
elastic pipeline tuned successfully, best parameters: {'model__alpha': 1.0,
'model__l1_ratio': 0.5}
Random Forest pipeline tuned successfully, best parameters: {'model__max_dep
th': 5, 'model__n_estimators': 200}
Gradient Boosting pipeline tuned successfully, best parameters: {'model__lea
rning_rate': 0.01, 'model__n_estimators': 50}
Support Vector Regressor pipeline tuned successfully, best parameters: {'mod
el__C': 0.1, 'model__gamma': 'scale'}
Best model for feature set ['Brittleness Ratio']: linear with R^2: 0.0063993
94939264558
linear pipeline trained successfully (no tuning parameters)
ridge pipeline tuned successfully, best parameters: {'model__alpha': 0.1}
lasso pipeline tuned successfully, best parameters: {'model__alpha': 0.001}
elastic pipeline tuned successfully, best parameters: {'model__alpha': 0.00
```

```

1, 'model__l1_ratio': 0.1}
Random Forest pipeline tuned successfully, best parameters: {'model__max_depth': 5, 'model__n_estimators': 200}
Gradient Boosting pipeline tuned successfully, best parameters: {'model__learning_rate': 0.01, 'model__n_estimators': 50}
Support Vector Regressor pipeline tuned successfully, best parameters: {'model__C': 0.1, 'model__gamma': 'auto'}
Best model for feature set ['TOC (%)']: Random Forest with R^2: 0.04643756589658876
linear pipeline trained successfully (no tuning parameters)
ridge pipeline tuned successfully, best parameters: {'model__alpha': 10.0}
lasso pipeline tuned successfully, best parameters: {'model__alpha': 1.0}
elastic pipeline tuned successfully, best parameters: {'model__alpha': 1.0, 'model__l1_ratio': 0.5}
Random Forest pipeline tuned successfully, best parameters: {'model__max_depth': 5, 'model__n_estimators': 200}
Gradient Boosting pipeline tuned successfully, best parameters: {'model__learning_rate': 0.01, 'model__n_estimators': 50}
Support Vector Regressor pipeline tuned successfully, best parameters: {'model__C': 0.1, 'model__gamma': 'auto'}
Best model for feature set ['Vitrinite Reflectance (%)']: linear with R^2: -0.0200429419452004

```

```

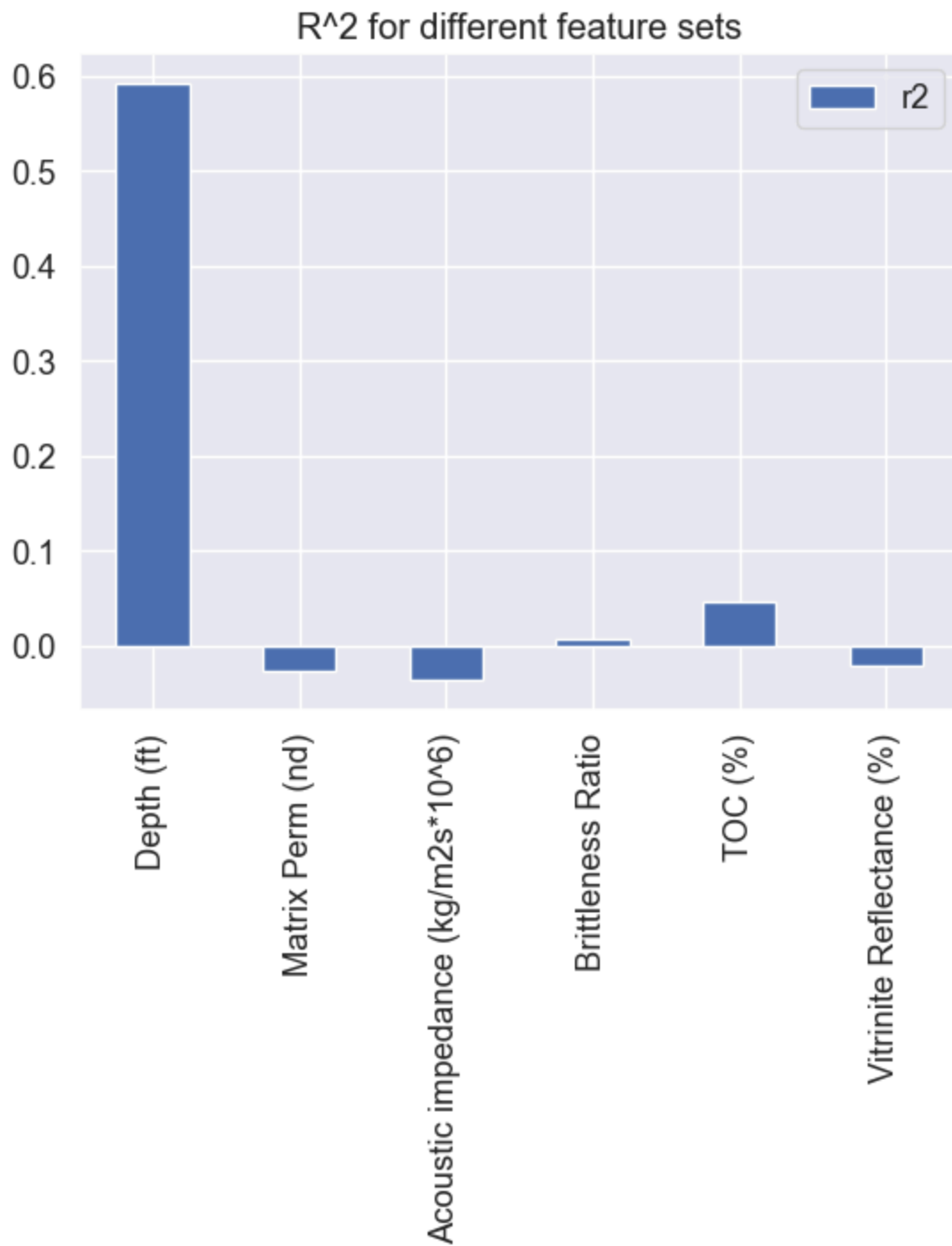
Out[24]: {'Depth (ft)': {'r2': 0.5926121023161295,
                        'mse': np.float64(2.1830910395184344),
                        'model': 'Support Vector Regressor'},
          'Matrix Perm (nd)': {'r2': -0.026183029275428815,
                              'mse': np.float64(5.499061186779504),
                              'model': 'linear'},
          'Acoustic impedance (kg/m2s*10^6)': {'r2': -0.035675584897860046,
                                                'mse': np.float64(5.549929445849735),
                                                'model': 'lasso'},
          'Brittleness Ratio': {'r2': 0.006399394939264558,
                                'mse': np.float64(5.324460029618763),
                                'model': 'linear'},
          'TOC (%)': {'r2': 0.04643756589658876,
                     'mse': np.float64(5.1099053686860785),
                     'model': 'Random Forest'},
          'Vitrinite Reflectance (%)': {'r2': -0.0200429419452004,
                                         'mse': np.float64(5.466157976574463),
                                         'model': 'linear'}}

```

```

In [25]: # plot metrics on best features
best_features_df = pd.DataFrame(best_features).T
# plot
best_features_df.plot(kind='bar', y='r2', title='R^2 for different feature s
plt.show()

```



In [26]: `best_features_df`

Out [26] :

	r2	mse	model
Depth (ft)	0.592612	2.183091	Support Vector Regressor
Matrix Perm (nd)	-0.026183	5.499061	linear
Acoustic impedance (kg/m2s*10^6)	-0.035676	5.549929	lasso
Brittleness Ratio	0.006399	5.32446	linear
TOC (%)	0.046438	5.109905	Random Forest
Vitrinite Reflectance (%)	-0.020043	5.466158	linear

In []: